# Kamień Milowy I – Analiza i Projekt Systemu

Aplikacja do zarządzania zadaniami (Todo App)

Bartłomiej Karkoszka Nazwisko2 Nazwisko3

Grupa: GRUPA

Przedmiot: MASI

## 1 Cel i zakres systemu

### 1.1 Cel

Celem systemu jest umożliwienie użytkownikom efektywnego zarządzania zadaniami osobistymi i zespołowymi poprzez intuicyjny interfejs webowy. System pozwala na tworzenie, edycję, śledzenie postępu oraz organizację zadań z uwzględnieniem priorytetów i terminów realizacji.

### 1.2 Zakres

System obejmuje:

- rejestrację i uwierzytelnianie użytkowników (z wykorzystaniem Keycloak),
- pełne operacje CRUD na zadaniach (tworzenie, odczyt, edycja, usuwanie),
- kategoryzację zadań według statusu (Do zrobienia, W trakcie, Zrobione) i priorytetu (Niski, Średni, Wysoki),
- widok listy zadań z filtrowaniem i wyszukiwaniem,
- widok tablicy Kanban z kolumnami odpowiadającymi statusom,
- panel główny (dashboard) z podsumowaniem statystyk zadań,
- przypisywanie terminów realizacji do zadań.

# 2 Aktorzy systemu

| **Aktor** | Opis |
|---|---|
| **Użytkownik** | Osoba korzystająca z systemu – tworzy, edytuje, przegląda i usuwa zadania. Posiada konto w systemie. |
| **Gość** | Osoba nieposiadająca konta lub niezalogowana – może jedynie przejść do strony logowania lub rejestracji. |
| **Keycloak** | System zewnętrzny odpowiedzialny za uwierzytelnianie i autoryzację użytkowników. |

# 3 Diagram przypadków użycia

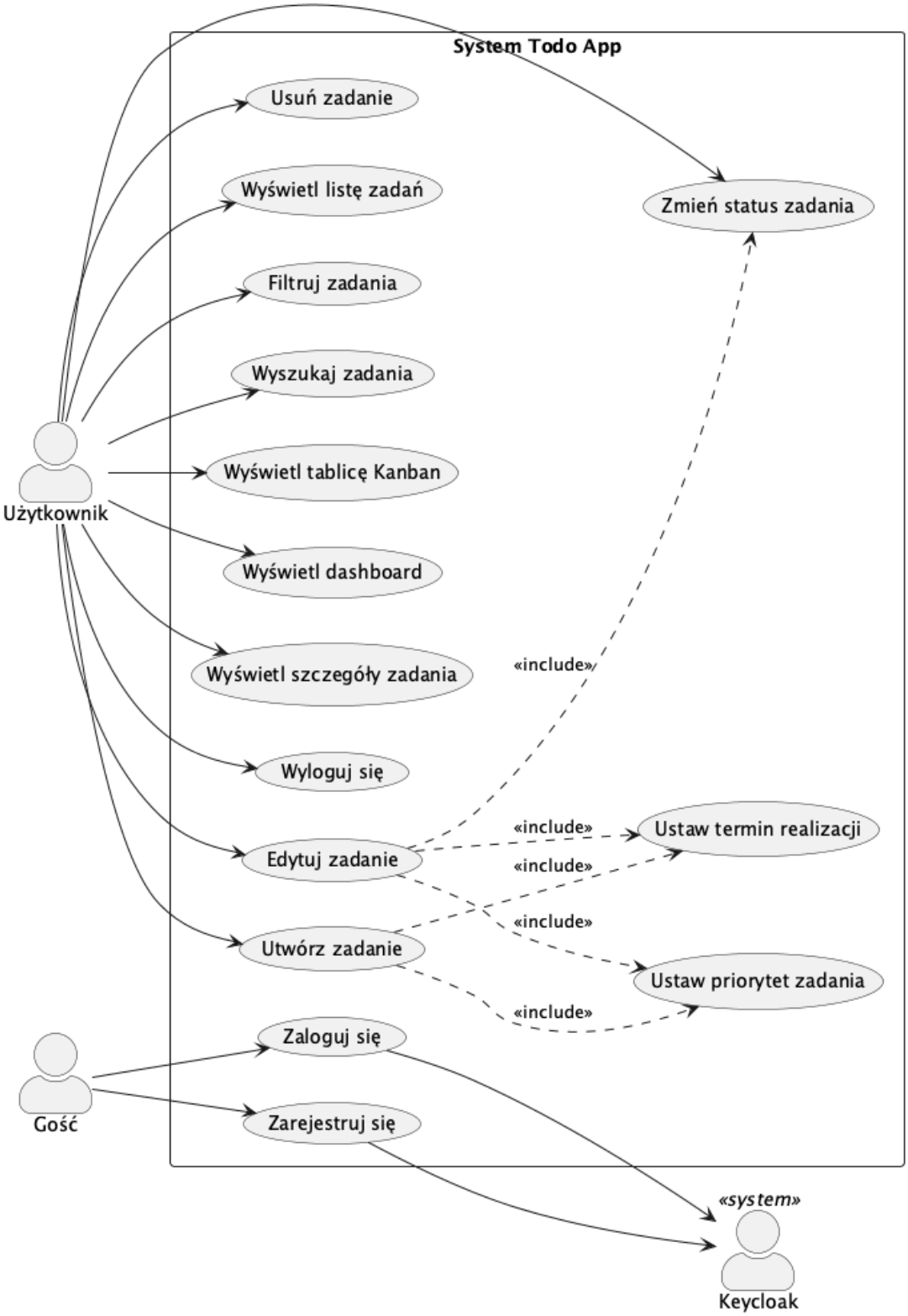



Diagram przedstawia następujące przypadki użycia:

- **Gość** może zalogować się lub zarejestrować (oba przypadki delegowane do Keyc-

loak).

- **Użytkownik** po zalogowaniu ma dostęp do pełnej funkcjonalności: zarządzania zadaniami, filtrowania, wyszukiwania, widoku Kanban i dashboardu.
- Tworzenie i edycja zadania zawierają (`«include»`) ustawianie priorytetu i terminu realizacji.

# 4 Wymagania funkcjonalne

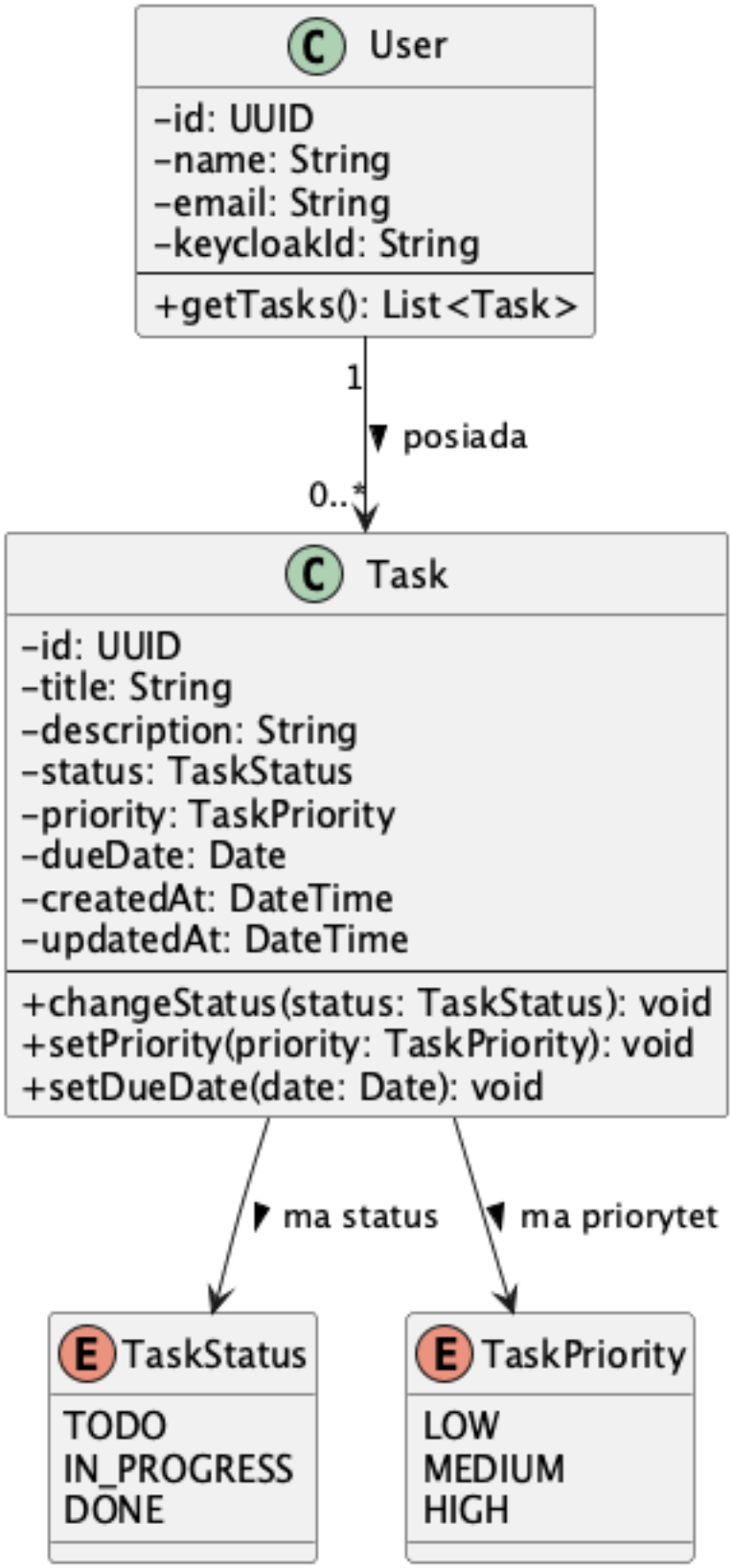
| Wymaganie | Priorytet |
|---|---|
| System umożliwia rejestrację nowego użytkownika poprzez Keycloak | Wysoki |
| System umożliwia logowanie użytkownika poprzez Keycloak | Wysoki |
| System umożliwia wylogowanie użytkownika | Wysoki |
| System umożliwia tworzenie nowego zadania z tytułem, opisem, priorytetem i terminem realizacji | Wysoki |
| System umożliwia edycję istniejącego zadania | Wysoki |
| System umożliwia usuwanie zadania | Wysoki |
| System umożliwia wyświetlanie listy wszystkich zadań użytkownika | Wysoki |
| System umożliwia filtrowanie zadań według statusu (TODO, IN_PROGRESS, DONE) | Średni |
| System umożliwia filtrowanie zadań według priorytetu (LOW, MEDIUM, HIGH) | Średni |
| System umożliwia wyszukiwanie zadań po tytule/słowie kluczowym | Średni |
| System wyświetla tablicę Kanban z kolumnami odpowiadającymi statusom zadań | Średni |
| System wyświetla panel dashboard z podsumowaniem liczby zadań w poszczególnych statusach | Niski |
| System umożliwia zmianę statusu zadania | Wysoki |
| System umożliwia przypisanie priorytetu do zadania | Wysoki |
| System umożliwia ustawienie terminu realizacji zadania | Średni |

# 5 Wymagania niefunkcjonalne

| Wymaganie | Kategoria |
|---|---|
| Interfejs użytkownika musi być responsywny (desktop, tablet, mobile) | Użyteczność |
| System musi obsługiwać przeglądarki Chrome, Firefox i Safari w najnowszych wersjach | Kompatybilność |

| Wymaganie | Kategoria |
|---|---|
| Czas odpowiedzi API nie powinien przekraczać 500ms dla operacji CRUD | Wydajność |
| System musi wykorzystywać HTTPS do komunikacji klient–serwer | Bezpieczeństwo |
| Uwierzytelnianie musi być realizowane przez zewnętrzny serwer Keycloak (OAuth2/OpenID Connect) | Bezpieczeństwo |
| Hasła użytkowników nie są przechowywane bezpośrednio w systemie (delegacja do Keycloak) | Bezpieczeństwo |
| Frontend musi być zbudowany w oparciu o React 19 z TypeScript | Technologia |
| Backend musi udostępniać REST API | Technologia |
| Dane muszą być przechowywane w relacyjnej bazie danych (PostgreSQL) | Technologia |
| System powinien być łatwo wdrażalny za pomocą kontenerów Docker | Wdrożenie |
| Kod źródłowy musi być wersjonowany w repozytorium Git | Utrzymanie |
| System powinien obsługiwać jednocześnie co najmniej 50 użytkowników | Wydajność |

# 6 Wstępny diagram klas




Model dziedziny obejmuje następujące klasy:

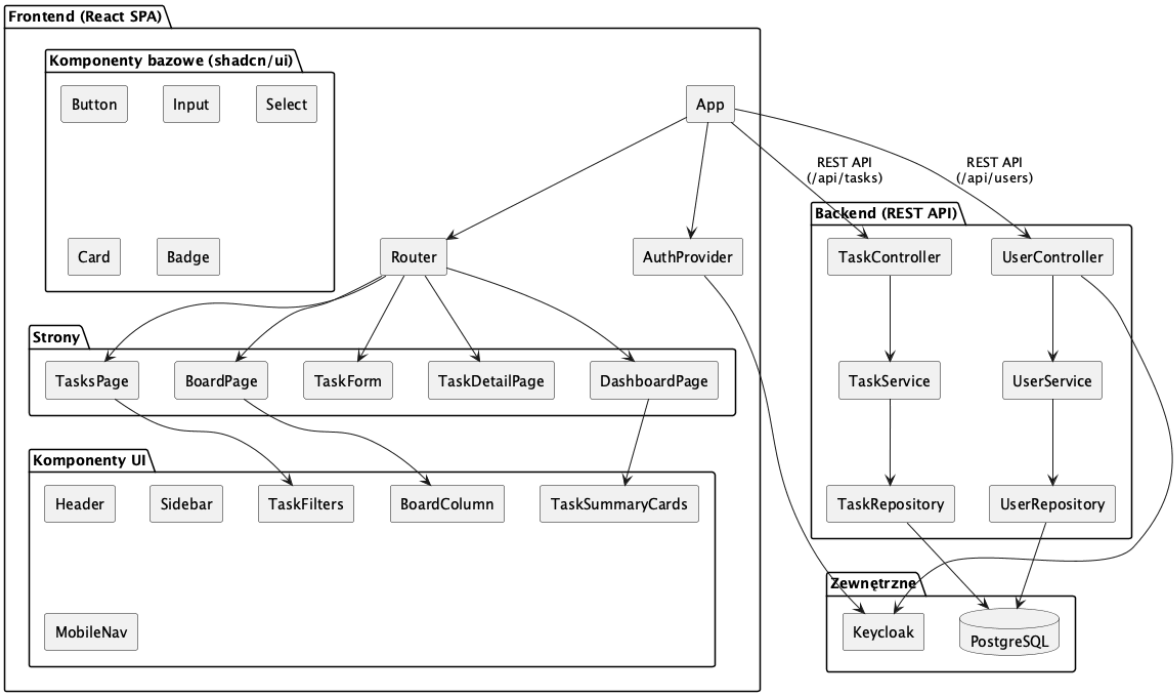
- **User** – reprezentuje użytkownika systemu, powiązanego z Keycloak poprzez pole

`keycloakId`.

- **Task** – podstawowa encja systemu, zawierająca tytuł, opis, status, priorytet, termin realizacji oraz znaczniki czasowe.
- **TaskStatus** – typ wyliczeniowy definiujący stany zadania: `TODO`, `IN_PROGRESS`, `DONE`.
- **TaskPriority** – typ wyliczeniowy definiujący priorytety: `LOW`, `MEDIUM`, `HIGH`.

Relacja: jeden użytkownik posiada wiele zadań (1 do 0..*).

# 7 Wstępny diagram komponentów

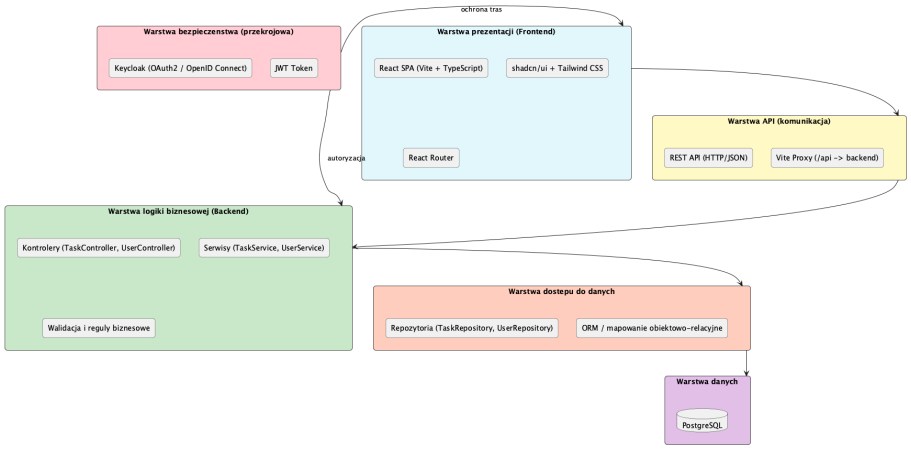



System składa się z trzech głównych pakietów:

- **Frontend (React SPA)** – aplikacja kliencka zawierająca strony (Dashboard, Tasks, Board), komponenty UI (Header, Sidebar, filtry) oraz komponenty bazowe (shadcn/ui).
- **Backend (REST API)** – warstwa serwerowa z kontrolerami, serwisami i repozytoriami obsługującymi logikę biznesową.
- **Systemy zewnętrzne** – baza danych PostgreSQL oraz serwer uwierzytelniania Keycloak.

Komunikacja między frontendem a backendem odbywa się przez REST API pod ścieżkami `/api/tasks` i `/api/users`.

# 8 Wstępny diagram warstw systemu




Architektura systemu oparta jest na modelu warstwowym:

1. **Warstwa prezentacji** – aplikacja React SPA z Tailwind CSS i React Router.
2. **Warstwa API** – komunikacja REST (HTTP/JSON) między frontendem a backendem, z wykorzystaniem proxy Vite w trybie deweloperskim.
3. **Warstwa logiki biznesowej** – kontrolery, serwisy oraz reguły walidacji.
4. **Warstwa dostępu do danych** – repozytoria i mapowanie obiektowo-relacyjne (ORM).
5. **Warstwa danych** – relacyjna baza danych PostgreSQL.

Warstwa bezpieczeństwa (Keycloak, tokeny JWT) jest warstwą przekrojową, przenikającą zarówno warstwę prezentacji (ochrona tras), jak i warstwę logiki biznesowej (autoryzacja żądań).

# 9 Słownik pojęć

| **Pojęcie** | Definicja |
|---|---|
| **Zadanie (Task)** | Podstawowa jednostka pracy w systemie, posiadająca tytuł, opis, status, priorytet i termin realizacji. |
| **Status** | Stan realizacji zadania: Do zrobienia (TODO), W trakcie (IN_PROGRESS), Zrobione (DONE). |
| **Priorytet** | Poziom ważności zadania: Niski (LOW), Średni (MEDIUM), Wysoki (HIGH). |
| **Tablica Kanban** | Wizualizacja zadań w kolumnach odpowiadających statusom. |
| **Dashboard** | Panel główny z podsumowaniem statystyk zadań użytkownika. |

# 10 Stos technologiczny

| **Warstwa** | Technologia |
|---|---|
| **Frontend** | React 19, TypeScript, Vite, Tailwind CSS v4, shadcn/ui, React Router v7 |
| **Backend** | Spring Boot (Java) lub Go – w trakcie decyzji |
| **Baza danych** | PostgreSQL |
| **Uwierzytelnianie** | Keycloak (OAuth2 / OpenID Connect) |
| **Konteneryzacja** | Docker, Docker Compose |
| **Kontrola wersji** | Git |